

파이썬으로 배우는 스택과 큐

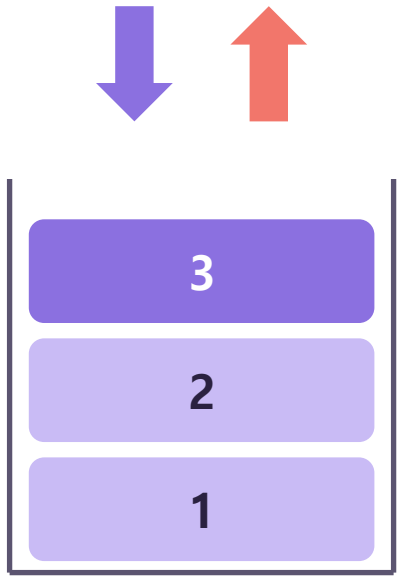
Stack · Queue — 기본 코드와 문제 접근법

CodeWithCathy

스택과 큐, 무엇이 다를까?

스택 Stack

LIFO · 나중에 들어온 게 먼저 나간다



넣기·빼기 모두 위에서만
push → 위에 쌓기
pop → 위에서 꺼내기

예) 접시 쌓기, Ctrl+Z, 뒤로 가기

큐 Queue

FIFO · 먼저 들어온 게 먼저 나간다



넣기는 뒤로, 빼기는 앞에서
append → 뒤에 줄 서기
popleft → 앞에서 나가기

예) 줄 서기, 프린터 대기열, 번호표

스택 기본 코드: 리스트 하나면 끝

Python

```python
stack = []

stack.append(1)     # push → [1]
stack.append(2)     # push → [1, 2]
stack.append(3)     # push → [1, 2, 3]

top = stack[-1]     # 맨 위 보기 → 3
x = stack.pop()     # 꺼내기 → 3, [1, 2]

if stack:           # 비어 있지 않을 때만
    print(stack[-1])
```

`append(x)`	맨 위에 넣기	O(1)
`pop()`	맨 위에서 꺼내기	O(1)
`stack[-1]`	꺼내지 않고 맨 위 보기	O(1)

! **빈 스택에서 pop()**

IndexError! 꺼내기 전에 if stack: 확인

큐 기본 코드: deque를 쓰자

Python

```python
from collections import deque

q = deque()
q.append(1)          # 뒤에 넣기 → [1]
q.append(2)          # → [1, 2]
q.append(3)          # → [1, 2, 3]

front = q[0]         # 맨 앞 보기 → 1
x = q.popleft()      # 앞에서 꺼내기 → 1

q = deque([4, 5])    # 리스트로 바로 만들기
```

왜 list.pop(0)은 안 될까?



← ← ← 남은 원소를 전부 한 칸씩 당김

`list.pop(0)` O(n)
`deque.popleft()` O(1)

원소 10만 개를 하나씩 앞에서 꺼내면 list는 최악 약 50억 번 이동, deque는 10만 번이면 끝나요

문제를 보고 무엇을 고를까?

스택을 떠올리는 신호

- 가장 최근 것을 취소 · 되돌리기
- 짝 맞추기 (열고 닫기)
- 직전 값과 비교하기
- 최근 값 두 개를 꺼내 계산
- 나보다 큰(작은) 다음 값 찾기

큐를 떠올리는 신호

- 먼저 온 순서대로 처리
- 대기열 · 줄 서기
- 맨 앞을 맨 뒤로 보내기
- 시간이 흐르면서 오래된 것부터 빠짐
- 가까운 곳부터 차례로 퍼지기 (BFS)

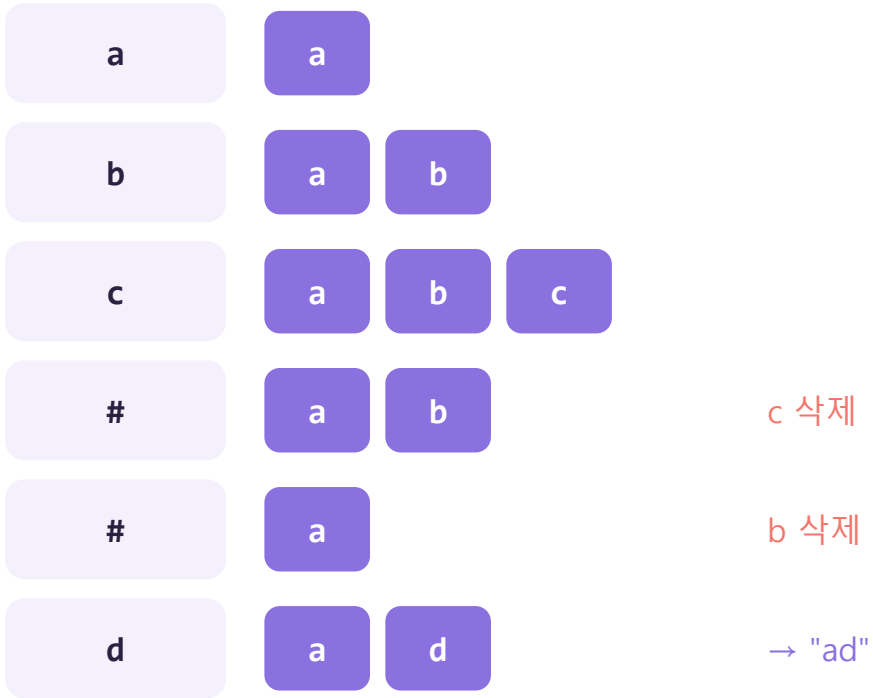
핵심 질문 **지금 꺼낼 것은 가장 최근 것? → 스택 가장 오래된 것? → 큐**

최근 것 취소하기

스택 패턴 ①

예제 **백스페이스 편집기**
"abc##d" → # 은 직전 글자 삭제 → "ad"

입력 글자 → 스택 상태 (오른쪽이 맨 위)




```python
def edit(text):
    stack = []
    for ch in text:
        if ch == "#":
            if stack:
                stack.pop()  # 최근 것 취소
        else:
            stack.append(ch)
    return "".join(stack)
```

이런 말이 보이면 되돌리기 · 취소 · 뒤로 가기 · 직전 것 지우기

최근 값 두 개를 꺼내 계산하기

스택 패턴 ②

예제 **후위 표기식 계산**

`"3 4 + 2 *"  →  (3 + 4) × 2 = 14`

토큰 → 스택 상태 (오른쪽이 맨 위)




```python
def calc(expr):
    stack = []
    for tok in expr.split():
        if tok in "+-*":
            b = stack.pop()   # 나중에 넣은 값
            a = stack.pop()
            if tok == "+": stack.append(a + b)
            elif tok == "-": stack.append(a - b)
            else: stack.append(a * b)
        else:
            stack.append(int(tok))
    return stack[-1]
```

이런 말이 보이면 연산 순서 · 계산기 · 수식 처리 · 중첩 구조

나보다 큰 다음 값 찾기

스택 패턴 ③

예제 **더 따뜻한 날까지 며칠?**
`[21, 19, 20, 24, 22]` → `[3, 1, 1, 0, 0]`

오늘 기온 → 대기 중인 날 (실제로는 인덱스 저장)



Python

```python
def warmer(temps):
    answer = [0] * len(temps)
    stack = []   # 아직 답을 못 찾은 날 (인덱스)
    for i, t in enumerate(temps):
        while stack and temps[stack[-1]] < t:
            j = stack.pop()
            answer[j] = i - j
        stack.append(i)
    return answer
```

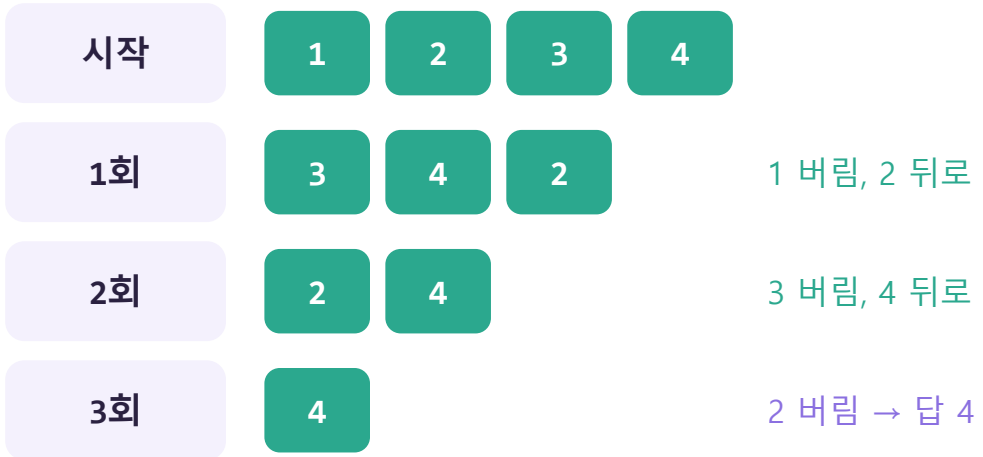
이런 말이 보이면 다음으로 큰(작은) 값 · 언제까지 유지되나 · 처음 바뀌는 순간

순서대로 처리하고, 맨 앞을 뒤로 보내기

큐 패턴 ①

예제 **카드 버리기 게임**
1~4번 카드: 맨 위는 버리고, 다음 장은 맨 아래로 → 마지막 카드?

왼쪽이 맨 앞(맨 위)



Python

```python
from collections import deque

def last_card(n):
    q = deque(range(1, n + 1))
    while len(q) > 1:
        q.popleft()              # 버리기
        q.append(q.popleft())    # 맨 뒤로
    return q[0]
```

이런 말이 보이면 차례대로 · 대기열 · 돌아가며 · 다시 줄 서기

시간이 흐르며 오래된 것부터 빠지기

큐 패턴 ②

예제 **최근 3000ms 안의 요청 수**
요청 시각 `[1, 100, 3001, 3002]` → `[1, 2, 3, 3]`

왼쪽이 가장 오래된 요청

t=1	1	→ 1개
t=100	1 100	→ 2개
t=3001	1 100 3001	→ 3개
t=3002	100 3001 3002	1 빠짐 → 3개

Python

```python
from collections import deque

def recent(times, k=3000):
    q = deque()
    result = []
    for t in times:
        q.append(t)
        while q[0] < t - k:   # 범위를 벗어난 요청
            q.popleft()       # 오래된 것부터 제거
        result.append(len(q))
    return result
```

이런 말이 보이면 시간 흐름 · 최근 N초 · 들어오고 나가기 · 시뮬레이션

스택 · 큐 문제 접근 4단계

1

꺼내는 순서

가장 최근? 가장 오래된?

```
최근 → stack
오래된 → deque
```

2

무엇을 넣을까

값? 인덱스? 묶음?

```
append(i)
append((t, v))
```

3

넣고 빼는 조건

언제 pop할지 정하기

```
while stack and
  조건: pop()
```

4

끝난 뒤 정리

빈 상태, 남은 원소

```
if stack:
남은 것 처리
```

특히 2번! 날짜 차이·위치가 필요하면 값 대신 인덱스를 넣어요

스택 vs 큐 한눈에 비교

구분	스택	큐
원칙	LIFO 나중에 온 게 먼저	FIFO 먼저 온 게 먼저
파이썬	`list`	`collections.deque`
넣기 / 꺼내기	`append() / pop()`	`append() / popleft()`
맨 끝 보기	`stack[-1]`	`q[0]`
대표 패턴	취소 · 짝 맞추기 · 다음 큰 값	순서 처리 · 뒤로 보내기 · 시간 흐름

주의 큐를 list.pop(0)으로 쓰면 O(n)! 큐는 항상 deque

오늘의 정리

1

스택은 list, 큐는 deque

append / pop / [-1] · append / popleft / [0]

2

고르는 기준은 "지금 꺼낼 것"

가장 최근 것이면 스택, 가장 오래된 것이면 큐

3

패턴으로 접근하기

스택: 취소 · 최근 값 계산 · 다음 큰 값 / 큐: 순서 처리 · 뒤로 보내기 · 시간 흐름